

Instructions: Submit both [Ms word+cpp] files in single zip file with naming convention RollNo_Sec_Dept_Name.zip on moodle.

Attempt all the questions.

CLO #02: Model an algorithmic solution for a given problem using OOP

[15+8+7 = 30]

Q1:

You are tasked with designing a real-time navigation system for autonomous vehicles in C++. This system must handle vehicles to efficiently manage intersections and the roads connecting them.

- Implement a class **Vehicle** that stores the vehicle's No (string), speed (float), company(string) and status (active/inactive) (bool).
- Implement a class **Road** that stores the road's Name (string), **lengthInMeters** (float), and dynamic array of vehicles.
- Implement a class **Intersection** that stores the intersection's ID (string) and a dynamic array of roads ('Road' objects) connecting to it.

Write a C++ code whether you would use composition or aggregation for the relationships between **Vehicle**, **Intersection** and **Road** based on your chosen associations.

Vehicle Class:

- 1) Add a **copy** constructor and destructors if required, think on it.
- 2) **addVehicle**(const Vehicle& v1)
- 3) **removeVehicle**(const std::string& id)
- 4) Implement a function to keep track of the total number of active vehicles and print them.

National University of Computer and Emerging Sciences

Chiniot-Faisalabad Campus

Road Class:

- 1) **calculateAverageSpeed()** const: that calculates the average speed of all active vehicles on the road.
- 2) **highSpeed()** that finds and returns the vehicle having maximum speed than others.

Note: use pointer notation in the above two functions (Half marks for array notation).

Intersection Class:

- 1) Implement **addRoad(const Road&)**
- 2) Implement **sortRoads()** const that returns a sorted list of all roads based on road length.

CLO #02: Model an algorithmic solution for a given problem using OOP

[15+20 = 35]

Q2:

Capital trust Bank needs to implement an account management system that can handle various types of bank accounts, specifically Current Accounts and Savings Accounts. The system should be designed using object-oriented principles, leveraging inheritance and polymorphism to manage account operations effectively. You are tasked with designing and implementing this system with the following requirements:

Create an abstract class called **Account** having data members:

- Private Account Number and Account Balance.
- Add suitable setter/getter for data.
- Add Debit(float), Credit(float) as member functions (Pure Virtual).
- Add Print() function (Virtual) o Override Print Debit and Credit functions according to derived classes.

Create a class called **CurrentAccount** i-e: CurrentAccount(is-a) Account having data members:

- Service Charges (To be charged during credit if account balance is less than min balance)
- Minimum Balance
 - Override print() as created in above class which displays:
- Account Number, Account Balance, Minimum Balance, Service Charges
- Modify the definition of the print() so that it displays a suitable message containing above info.
- Similarly override credit(float), debit(float) functions such that credit(float) simply add amount to the Account Balance and debit(float) checks if the amount to be debited is within the range of Account Balance, and further if the amount is account balance is less than min balance standard charges would also be deducted.

Create a class called **SavingAccount** i-e: SavingAccount (is-a) Account having data members:

Interest Rate.

- Override print() as created in parent class which displays:
- Account Number, Account Balance, Interest Rate
- Modify the definition of the print() so that it displays a suitable message containing above info.

- Similarly override credit(float), debit(float) functions such that credit(float) simply add amount to the Account Balance and debit(float) checks if the amount to be debited is within the range of Account Balance.

In main function:

- Create an array of **Account** type **Pointers**, of size 3. Assign Account object to index 0, CurrentAccount object to index 1 and SavingAccount object to index 2.
- Have you encountered any problem? Report the problem and change the size and elements of the array accordingly
- Now Credit and Debit the CurrentAccount and SavingAccount in main function

Although things seem to be fine on the surface, there is a problem in the program we just wrote. To observe this problem, we must add destructors for all classes. Paste the following inline definitions of the destructors in their corresponding classes, execute the program and paste the output below. Your destructors should delete the child classes first before destroying parent class. There should not be any memory leaks.

Part (B): Operator Overloading [Marks: 20]

Implement a class called **Quadratic**. The class will have three data members:

//It'll form a number as ax^2+bx+c

You have to implement default constructor, overloaded constructor, copy constructor, destructor and overload the operators +, -, *, <<, >>, ==, !=, = as described below:

- 1) - => Subtract one quadratic object from other (friend function)
- 2) * => Multiply a constant with Quadratic object (Member Function only)
- 3) >> => Input a quadratic object and stores it in reverse order
- 4) << => Display a quadratic object in reverse order
- 5) != => In-equality operator (Member Function only)
- 6) = => Assignment operator (Member Function only)
- 7) [] => Access coefficients using an index (0 for a, 1 for b, 2 for c).

CLO #04: Apply good programming practices

Q3: Question statement

[30]

You've been assigned to develop a **generic task management system** for a project management company. The system should be capable of managing tasks of various types, such as development tasks, design tasks, and testing tasks. It should support functionalities like adding, removing, searching, and sorting tasks within the system. Additionally, it should handle exceptions for invalid operations and utilize a **fixed-size array** for efficient storage.

Requirements: Class Template TaskManager

Develop a template class named **TaskManager** capable of storing and managing tasks of any data type in a fixed-size array.

Support the following operations:

Add Task at Index: Add a task at a specified index in the array. If the index is out of bounds or the slot is already occupied, throw an InvalidOperationException.

Remove Task at Index: Remove a task from a specified index in the array. If the index is out of bounds or the slot is empty, throw an InvalidOperationException.

Search Task: Search for a task by a given attribute (e.g., task ID, name, category, priority). If the task is not found, throw a TaskNotFoundException.

0069

National University of Computer and Emerging Sciences

Chiniot-Faisalabad Campus

Sort Tasks: Sort the tasks based on a provided comparator function. Implement the sorting algorithm independently without utilizing any library sorting functions.

Display Tasks: Print all tasks along with their attributes (task ID, name, category, priority).

Function **Template searchByAttribute**

Implement a function template named `searchByAttribute`, which takes an array of tasks, a search key (searching attribute), and a function pointer for comparing the search key with task attributes. It should return the found task, handling exceptions gracefully for tasks not found.

Exception Classes: Create **custom** Exception Classes:

- Create custom exception classes:
 - **TaskNotFoundException** and **InvalidOperationException**
- Each exception class should inherit from exception and override the `what()` method to provide a meaningful error message.

Main Function:

- Instantiate instances of `TaskManager` for different task types (e.g., Development, Design, Testing).
- Demonstrate the functionality by performing operations such as adding, removing, searching, printing, and sorting tasks.
- You are required to call the display function after calling each function (e.g. adding, removing, and sorting).
- Handle exceptions gracefully and print appropriate error messages.
- **Implementation Details:**
 - Utilize a fixed-size array internally within `TaskManager` for storing tasks efficiently.
 - Ensure proper memory management to avoid memory leaks.
 - Implement a custom sorting algorithm without relying on any library functions.
 - Write clean, readable, and well-documented code to enhance maintainability and understandability.

Task Attributes:

Each task in the system should have the following attributes:

- **ID:** A unique identifier for the task (e.g., `int taskID`).
- **Name:** The name of the task (e.g., `string name`).
- **Category:** The category of the task (e.g., Development, Design, Testing) (e.g., `string category`).
- **Priority:** The priority level of the task (e.g., High, Medium, Low) (e.g., `enum Priority`).

Comparator Functions:

Implement comparator functions to search and sort by the following attributes:

Category: Compare two tasks based on their category.

Priority: Compare two tasks based on their priority level.